Reviewer Pack — Additive Energy of Fourier Coefficients and ACC⁰ Circuit Siz…

Entry 2088c7f83a48 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 08:19:59 UTC

Additive Energy of Fourier Coefficients and ACC⁰ Circuit Size

Entry ID: `2088c7f83a48` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 08:19:59 UTC

1. Conjecture

Field A (mathematical branch): Additive Combinatorics **Field B** (complexity object): ACC⁰ Circuit Size

Statement:

> For every Boolean function f: {0,1}^n → {0,1} in P, the additive energy E(f) of its Fourier coefficients satisfies E(f) ≥ 2^{Ω(n)}, whereas for ACC⁰ circuits computing f, E(f) ≤ $2^{\{O(\log^2 n)\}}$.

Rationale (proposer's reasoning):

> Additive energy captures the structure of Fourier coefficients, which reflect the correlation of f with low-degree polynomials. ACC⁰ circuits, constrained by bounded-depth, exhibit limited structure, leading to lower additive energy. This could expose a gap between P and ACC⁰ via Fourier-analytic methods.

Taxonomy category: `ACC_SIPSER` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `33ae0b0f5316ba76`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def fast_walsh_hadamard_transform(f):
    n = len(f)
    if n == 1:
        return f
    even = fast_walsh_hadamard_transform(f[0::2])
    odd = fast_walsh_hadamard_transform(f[1::2])
    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]
    return result

def add(a, b):
    return a + b

def multiply(a, b):
    return a * b

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 10
    f = [random.randint(0, 1) for _ in range(2**n)]

    transformed_f = fast_walsh_hadamard_transform(f)
    E_f = sum(abs(transformed_f[i] * transformed_f[j]) for i in range(len(transformed_f)) for j in r

    # Placeholder for ACC⁰ function check
    is_acc0 = False

    if is_acc0:
        conjecture_holds = E_f <= 2**(O(log**2 n))
    else:
        conjecture_holds = E_f >= 2**(Ω(n))

    return {
        "metric_name": "Additive Energy",
        "metric_value": E_f,
        "instances_tested": len(f),
```

```
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(3, 6)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_E_f = sum(r["metric_value"] for r in results) / len(results)
    std_E_f = math.sqrt(sum((r["metric_value"] - mean_E_f)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_E_f} std={std_E_f} support_fraction={support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_E_f} std={std_E_f} support_fraction={support_fraction
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1fe85aa.py", line 69, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1fe85aa.py", line 47, in run_trial
    transformed_f = fast_walsh_hadamard_transform(f)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c1fe85aa.py", line 26, in fast_walsh_ha
    v = f[j + m]
        ~^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed during execution, preventing evaluation of the conjecture's validity. | next: Debug the fast_walsh_hadamard_transform implementation to fix index out of range error

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	52476
2	preregistration	ollama_remote	qwen3:8b	0	0	32160
3	novelty	ollama_remote	qwen3:8b	0	0	27965
4	novelty	ollama_remote	qwen3:8b	0	0	20325
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11562
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8454
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7320
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7798
9	judge	ollama_remote	qwen3:8b	0	0	21841

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 189901 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2088c7f83a48.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2088c7f83a48.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2088c7f83a48.tar.gz` (if generated)